

ALASCA — Architecture logicielles avancées pour les systèmes cyber-physiques autonomiques

© **Jacques Malenfant**

Master informatique, spécialité STL – UFR 919 Ingénierie

Sorbonne Université
Jacques.Malenfant@lip6.fr

Cours 8

De l'entité adaptable — conception des actionneurs et méthodologie de développement

Objectifs pédagogiques du cours 8

- Comprendre les problématiques liées à la *fonctionnalité d'adaptation*, à la conception des *interfaces actionneurs* d'une entité adaptable).
- Introduire des *techniques d'implantation* des fonctions d'adaptation et des actionneurs.
- Établir des *principes de conception et de modélisation* qui vont servir de base à une méthodologie et un processus de développement logiciel spécifique aux entités adaptables (conception, développement, test, validation et déploiement).
- Développer le principe d'une *intégration de la simulation au sein d'un modèle composants* pour les CPCS en vue de soutenir une méthodologie de développement propre à ces derniers.
- Développer des *savoir-faire* dans la conception et la mise en œuvre des éléments gérés en informatique autonome.

Plan

- 1 Opérations d'adaptation
- 2 Les actionneurs et leurs interfaces
- 3 Test et validation des entités adaptables
- 4 Simulateurs DEVS et composants BCM4Java
- 5 Aide au déploiement

Problématiques

- Bien qu'elles provoquent des changements d'états, la conception de la fonction adaptation est trop souvent abordée sous les hypothèses *simplificatrices* :
 - les opérations d'adaptation sont *élémentaires* et *orthogonales*,
 - elles se réalisent *individuellement* sur *chaque* entité adaptable et
 - elles *n'interfèrent pas* avec le fonctionnement de l'élément lui-même, des autres éléments et du système en général,en réalité *fausses* pour tout système *réaliste* !
- Distinguons (à nouveau) opérations d'adaptation (implantation) et leur visibilité externe (interfaces), les questions se posant sont :
 - Quelles adaptations sont nécessaires et quelles opérations offrir ?
 - Quelles propriétés adaptables affectent-elles et comment ?
 - Comment les mettre en œuvre et les rendre robustes ?
 - Comment en tirer des interfaces cohérentes et flexibles ?
 - Comment gérer les adaptations multiples, dépendantes entre elles et réparties entre plusieurs entités ?

Opérations d'adaptation *élémentaires*

- Les opérations d'adaptation provoquant un changement d'état de l'entité, il devient généralement nécessaire de les exécuter en *exclusion mutuelle* avec les services de cette entité.
- Du coup, la caractérisation de ces opérations par rapport à l'exclusion mutuelle devient essentielle.
- Les opérations d'adaptation *élémentaires* se caractérisent par :
 - leur *complétude*, formant un passage direct d'un état de fonctionnement *cohérent* à un autre état *cohérent* ;
 - d'où leur nécessaire *non interruptibilité*, car leur interruption laisserait l'entité dans un état de fonctionnement intermédiaire incohérent.
- Une opération élémentaire doit être de courte durée, sauf cas particulier, car l'exclusion mutuelle nécessite de suspendre l'entité adaptée pendant toute la durée de son exécution.
 - Ex.: modification du nombre de threads dans un pool de threads ou d'un seuil dans un filtre sur des données.

Opérations d'adaptation composites

- D'autres opérations sont trop complexes, trop longues ou impactant d'autres entités, pour être élémentaires.
Ex.: modifier le nombre de MV d'une application.
- Elles peuvent s'appuyer sur des opérations élémentaires, en les *composant* pour former une adaptation *composite* cohérente.
- Elles se caractérisent donc par leur :
 - *durée*, trop longue pour simplement suspendre l'exécution de l'entité pendant *toute leur exécution*, une gestion plus fine de l'exclusion mutuelle devenant nécessaire ;
 - *parallélisabilité*, se décomposant en plusieurs (sous-)adaptations ou calculs qui peuvent se faire (partiellement) en parallèle ;
 - *non indépendance*, pouvant devoir être entrelacées avec des opérations d'adaptation sur d'autres entités pour respecter des invariants locaux ou globaux du système.

Ces caractéristiques sont déjà visibles dans l'exemple d'adaptation à la bande passante du réseau sans fil.

Atomicité et adaptations atomiques

- Que faire si une opération d'adaptation nécessite une interruption ?

Exemple : dans le cas de l'échange de données par réseau sans fil, que faire si la bande passante redevient forte (resp. faible) pendant une adaptation ?

- Lors d'une interruption, distinguons l'état du système entre :
 - état *fatalement incohérent* où l'exécution ne peut reprendre sans erreur, nécessairement à éviter ;
 - *Atomicité* : propriété d'une séquence de modifications soit totalement réalisée soit pas du tout, grâce à un mécanisme de retour arrière (*rollback*).
 - ⇒ *opérations atomiques*, plutôt élémentaires ;
 - ⇒ *blocs d'instructions atomiques* au sein d'opérations plutôt composites.
 - état *partiellement incohérent* permettant une reprise mais après remise en cohérence.
 - utiliser exceptions et blocs *try/catch* pour remettre en cohérence ;
 - généralement moins coûteux que l'atomicité et possibilité d'une atteinte partielle des objectifs d'adaptation.

Supervision et coordination des opérations d'adaptation

- Les opérations d'adaptation composites peuvent être exécutées
 - en parallèle entre elles au sein d'une entité, ou
 - en parallèle avec des adaptations lancées sur d'autres entitésce qui nécessite une coordination d'ensemble, mais comment ?
 - Insertion d'*instructions intermédiaires* pour assurer le suivi de progression et la synchronisation :
 - outils de synchronisation (sémaphores, barrières de synchronisation, jointures, etc.) ;
 - échanges de messages, signaux ou données (approches de type protocoles), possiblement pair-à-pair.
- ⇒ Concepts de *plan* et de *moteur d'exécution* de plans.
- Deux options sont possibles :
 - 1 une approche maître/esclave imposant une *supervision* centralisée pour ordonnancer ces opérations, par exemple en les appelant dans l'ordre et en contrôlant leur progression ;
 - 2 une approche pair-à-pair où les opérations sont conçues de telle manière à coopérer directement entre elles.

Plan

- 1 Opérations d'adaptation
- 2 Les actionneurs et leurs interfaces**
- 3 Test et validation des entités adaptables
- 4 Simulateurs DEVS et composants BCM4Java
- 5 Aide au déploiement

Mise en œuvre des actionneurs

- L'actionneur est aux opérations d'adaptation ce que le capteur est aux opérations de mesure :
 - un actionneur utilise une opération d'adaptation implantée au sein de l'entité adaptable pour réaliser une action sur l'entité ;
 - il est rendu visible par une interface actionneur.
- Comme pour les capteurs, la grande variété des cas rend difficile toute tentative d'élaboration d'une méthodologie générale.
- Nous nous concentrons sur les concepts suivants :
 - relation entre opérations d'adaptation et fonctionnement de l'entité adaptable,
 - coopération et coordination entre actionneurs,
 - interruptibilité, traitement des erreurs et exécution atomique.

Adaptation versus exécution des entités adaptables

- Sauf pour des cas particuliers, adapter une entité doit se faire au moins partiellement en exclusion mutuelle avec l'exécution de ses services fonctionnels.
- La conception d'une entité adaptable doit donc assurer cette exclusion mutuelle, ce qui a des conséquences dont il faut limiter l'impact des sur l'exécution « normale ».
 - L'accès exclusif global, par exemple en utilisant des clauses `synchronize` systématiques sur les méthodes, impliquerait potentiellement la sérialisation totale de toute exécution de code sur l'entité, y compris entre ses services « normaux ».
 - Prévoir plutôt des sections critiques correctement délimitées mises en œuvre par des verrous « read/write locks », par exemple.
- Bien cerner les sections critiques des opérations d'adaptation pour obtenir l'exclusion mutuelle strictement *nécessaire* permet
 - d'augmenter la disponibilité et
 - d'améliorer la performance globale de l'entité.

Coopération entre adaptations conjointes

- Exemple Molène : illustration des difficultés soulevées par l'adaptation de plusieurs entités adaptables :
 - adaptations intégrant ou retirant la compression/décompression dans les chaînes d'émission/réception de données.
- Outre l'exclusion mutuelle, l'interaction entre les deux entités et l'adaptation à réaliser posent les problèmes suivants :
 - ❶ Comment entrelacer les étapes de mise en place de l'adaptation entre les deux entités ?
 - ❷ Comment faire en sorte que l'émetteur ne commence pas à compresser les données envoyées avant que le récepteur ne soit prêt à les décompresser (et vice versa) ?
 - *Par souci d'indépendance et d'abstraction, chaque entité réalise ses propres adaptations, mais les deux doivent coopérer pour que les échanges de données se déroulent correctement en tout temps.*
 - La composition de ces opérations locales forme une opération inter-entités *atomique* qui doit être réalisée sans s'interrompre, complètement ou pas du tout.
 - ❸ Subsidiairement, quelles étapes peuvent être exécutées en parallèle pour diminuer la durée de l'adaptation ?

MolèNE : coopération entre opérations d'adaptation

- Pour l'activation de la compression, les deux entités vont devoir effectuer les opérations suivantes :
 - ❶ Créer et initialiser les éléments logiciels de compression/décompression
 - ❷ Acquérir l'accès exclusif à chacun des composants en interrompant les échanges de données
 - ❸ Intégrer les éléments logiciels de compression/décompression dans chacun des composants
 - ❹ Basculer dans le deux composants vers la chaîne d'émission avec compression/décompression
 - ❺ Relâcher l'accès exclusif à chacun des composants pour reprendre les échanges de données
- Forme de protocole en deux phases, calcul puis basculement (*commit*), comme pour les transactions dans les BD.
- Synchronisations avant d'interrompre les échanges, et à nouveau avant de reprendre, les autres étapes pouvant être exécutées en parallèle.

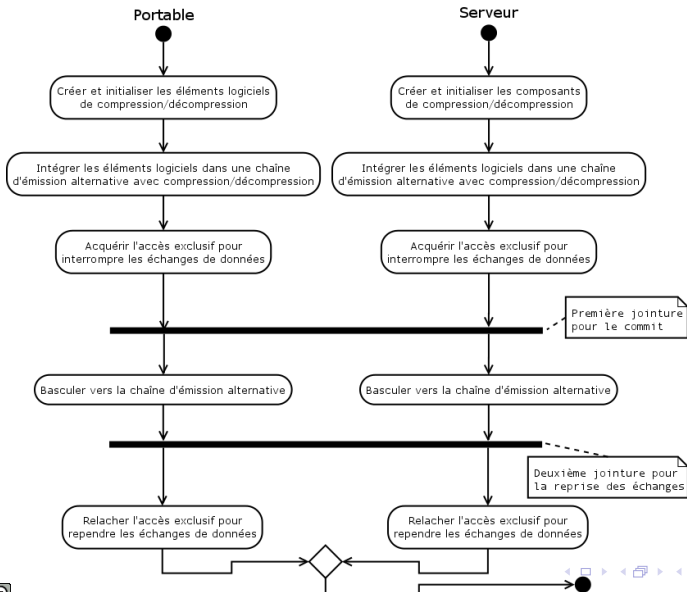
Protocoles de coopération entre actionneurs

Protocole

Ensemble des conventions nécessaires pour faire coopérer des entités indépendantes *i.e.*, assurer la cohérence entre leurs exécutions ainsi qu'établir et entretenir des échanges d'informations entre elles.

- 1 Un protocole peut être modélisé de différentes manières avec différents outils, par exemple :
 - Automates, les états marquant la progression dans le protocole et les transitions indiquant les prochains événements et actions possibles.
C'est l'approche classique dans les domaines des réseaux et habituelle en programmation répartie, surtout pour le pair-à-pair.
 - En UML, les diagrammes d'activité permettent d'exprimer le déroulement des comportements et des échanges entre différentes entités coopérantes.
C'est une approche plus couramment utilisée en modélisation et développement logiciel.

Modélisation des activités d'adaptation pour la compression



Mise en œuvre

- Les deux *jointures* du diagramme d'activités précédent peuvent se réaliser de plusieurs manières :
 - Utiliser une entité de supervision recevant les notifications des deux entités et leur envoyant des signaux pour les coordonner.
 - Utiliser conjointement un même synchroniseur.
 - Lier les deux entités directement en pair-à-pair pour qu'elles s'échangent directement des signaux.
- Ici, la solution la plus simple et peut-être la plus élégante est l'utilisation du synchroniseur *barrière de synchronisation* :
 - elle est créée puis liée à n processus devant se synchroniser ;
 - chaque processus appelle une méthode `wait` sur la barrière ;
 - les processus ayant appelé `wait` attendent jusqu'à ce que tous les autres l'aient fait et alors tous sont libérés et poursuivent leur exécution.
- Plus généralement, problématique de génie logiciel ouverte :
Comment programmer et faire apparaître dans des interfaces les synchronisations et la coordination nécessaires entre actionneurs d'entités adaptables de manière générale ?

Interruptibilité, traitement des erreurs et transactions

- Outre l'exclusion mutuelle, la robustesse exige de toujours assurer le retour à un état de fonctionnement sain.
- Cela passe par la mise en œuvre de mécanismes de type transactionnels, tâche parmi les plus complexes en informatique.
- Une opération d'adaptation composite robuste devrait donc :
 - Déterminer des blocs transactionnels chapeautant les sections critiques au cours de l'exécution desquels toute erreur ou interruption entraînera un retour à un état cohérent.
 - En cas d'interruption dans un état fatalement incohérent, il faut assurer le retour (*rollback*) de l'entité touchée et de toutes les autres entités impliquées à l'état initial avant le début de l'adaptation ;
 - En cas d'interruption dans un état partiellement incohérent :
 - traiter et propager toute interruption ou exception levée de manière à ramener l'entité touchée et les autres entités impliquées dans un état cohérent, prête à fonctionner ;
 - chaque cas étant particulier, il s'agit de solution *ad hoc* à développer individuellement.

Méta-propriétés des actionneurs (spécifier et documenter)

- 1 Effets sur les propriétés adaptables : propriétés affectées, inertie, y compris de manière croisée (entre actionneurs).
- 2 Assertions : pré-conditions, post-conditions et invariants exigées, garanties ou maintenues.
- 3 Type et atomicité : élémentaire versus composite, exécutabilité comme tout ou partie d'une transaction d'adaptation.
- 4 Supervisabilité : moyens offerts pour suivre le progrès et intervenir pour les synchroniser et les coordonner.
- 5 Interruptibilité : possibilité ou non d'interrompre (volontairement) l'exécution de l'opération d'adaptation et ses conséquences.
- 6 Propriétés temporelles : durée, période minimale entre les appels, inertie (délai entre l'action et son effet).
- 7 Coûts en ressources : de toutes natures.

Plan

- 1 Opérations d'adaptation
- 2 Les actionneurs et leurs interfaces
- 3 Test et validation des entités adaptables**
- 4 Simulateurs DEVS et composants BCM4Java
- 5 Aide au déploiement

Problématique du test des CPCS

- Outre les erreurs de programmation classiques, il faut s'assurer que le code fournit les *bonnes réactions* aux actionneurs *selon les données courantes* des capteurs et ce dans la *bonne temporisation* pour produire une *bonne trajectoire*.
- Idéalement, les tests se feraient donc toujours en contexte de déploiement, mais cela n'est que partiellement possible, pour des raisons de disponibilité, de coûts ou de sécurité.
- Dans une approche de tests dirigés par les modèles, tester un CPCS *hors déploiement* requiert d'avoir des modèles préalablement validés servant d'oracles pour :
 - 1 Fournir des entrées *réalistes*, remplaçant les données capteurs.
 - 2 S'assurer que chacune des sorties sur les actionneurs est *correcte* compte tenu des entrées.
 - 3 S'assurer que la *trajectoire* des sorties et de l'état au sens de la théorie des systèmes et du contrôle sont conformes aux spécifications.

Comment vérifier et valider les CPCS ?

- Au-delà du test pour la mise au point du code, les CPCS doivent aussi être vérifiés et validés, ce qui exige des tests méthodiques et complets.
- Outre les difficultés rencontrées en test de mise au point, assurer une *couverture complète* des scénarios possibles est beaucoup plus complexe pour les CPCS que les applications classiques.
- Pour couvrir la variété des situations possibles, il faut :
 - des *expérimentations contrôlées*
 - faisant varier *systématiquement* les configurations initiales
 - et évoluer les valeurs des propriétés d'adaptation
 - sur *l'entièreté de leurs domaines*,

ce qui est très difficile (voire impossible) et généralement très coûteux à faire sur des *systèmes réels en contexte de déploiement*.

Tests dirigés par les modèles en simulation

- Les approches classiques fondées sur les cas de tests par entrées/sorties ponctuelles sont insuffisantes pour les CPCS.
 - Il ne s'agit pas simplement de vérifier l'exactitude des couples entrées/sorties.
 - Il faut s'assurer que l'*enchaînement* bien *temporisé* des valeurs de capteurs et de sorties sont *conformes à la réalité* et produisent une *trajectoire correcte*.
- À défaut de pouvoir s'en assurer dans le monde physique, l'approche proposée se fonde sur la *simulation*, par transformation des modèles de comportement en *modèles de simulation*.
 - En tant que modèle *exécutable*, le simulateur calcule des valeurs d'états et des sorties réalistes en fonction d'entrées réalistes, ce qui permet dans un contrôle en boucle fermée
 - de fournir au contrôleur une valeur d'état réaliste pour le calcul de l'entrée contrôlée suivante ;
 - de construire la trajectoire suivie pour la comparer à celle attendue.
 - La simulation permet également d'explorer une grande variété de situations possibles par des campagnes systématiques.

Concept de co-simulabilité

- Question : comment produire des scénarios et tester de manière réaliste ?
 - Prédéfinir des trajectoires des entrées incontrôlées à simuler.
 - Faire calculer par simulation les trajectoires des valeurs d'entrées contrôlées, d'états et de sorties.
 - Connecter cette simulation avec le code pour ainsi fournir des enchaînements de données réalistes, comme si l'action des sorties calculées se produisaient dans le monde physique.
- *Cette idée peut être mise en œuvre par la co-simulation.*
 - La co-simulation consiste à simuler conjointement et faire interagir plusieurs phénomènes parallèles ; dans le cas présent il s'agit
 - 1 de simuler la plate-forme matérielle et l'environnement physique en interaction avec le logiciel (simulation SIL) puis
 - 2 de simuler uniquement l'environnement physique en interaction avec le logiciel déployé sur une plate-forme matérielle cible (simulation HIL).

Nota : SIL = software-in-the-loop, HIL = hardware-in-the-loop. Aussi MIL = model-in-the-loop ou simulation « pure ».

Simulation et architectures logicielles CPS : objectifs

- Chaque composant ayant un comportement autonome ou cyber-physique, il serait intéressant :
 - de modéliser *individuellement* leur comportement et leurs dépendances envers d'autres modèles ;
 - d'attacher ces modèles de comportement aux composants.
- Selon l'approche « composants », il devient alors possible :
 - d'implanter les composants individuellement ;
 - d'implanter des simulateurs SIL/HIL modulaires au sein de chaque composant puis connecter exécution du logiciel et simulation ;
 - de construire l'*architecture globale* de l'application en *assemblant* ses composants ;
 - de construire le *simulateur global* de l'application en *composant* les simulateurs modulaires individuels.

Les deux dernières phases étant intégrées dans une nouvelle forme d'assemblage de composants cyber-physiques.

Principe de co-simulabilité des entités adaptables

Principe de co-simulabilité

Toute entité adaptable doit être conçue et mise en œuvre de manière à pouvoir être co-simulée, d'abord pour la mettre au point puis pour la vérifier et la valider.

- L'objectif principal de cette co-simulabilité est de produire lors d'une exécution des valeurs de propriétés non-fonctionnelles correctes, cohérentes entre elles et cohérentes dans le temps.
- Un objectif qui peut aussi être très important dans certains cas est la simulation de certains aspects fonctionnels pour simplifier le déploiement des tests unitaires.

Ex.: simuler sur un ordinateur unique une application qui devrait être déployée en réparti.

- À titre de preuve de concept, nous allons examiner une intégration BCM4Java/DEVS.

Plan

- 1 Opérations d'adaptation
- 2 Les actionneurs et leurs interfaces
- 3 Test et validation des entités adaptables
- 4 Simulateurs DEVS et composants BCM4Java**
- 5 Aide au déploiement

Principes d'intégration de DEVS4Java/BCM

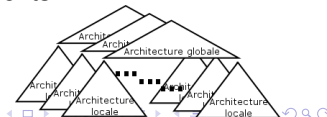
- Proposition d'une *unité de composition autonome*, le *composant cyber-physique*, soutenant un génie logiciel fondé sur leur co-simulabilité, et pour cela intégrant :
 - un modèle de comportement (automate hybride) *modulaire* et *déclaratif* (spécification, conception, etc.) et
 - un modèle de simulation *modulaire* et *exécutable* (mise au point, test, V&V, etc.),puis faire en sorte que l'assemblage de composants cyber-physiques intègre ces modèles.
- Cette intégration pose toutefois quelques contraintes :
 - Les simulateurs DEVS *internes* aux composants devront être composés par des modèles couplés *inter-composants*.
 - Cela implique une architecture de simulation explicitant ses liens avec les composants *i.e.*, description du déploiement des modèles sur et entre les composants.
 - Cela implique également des échanges entre simulateurs passant potentiellement par des connexions entre composants répartis, d'où une pénalité potentielle en performance à prendre en compte.

Contraintes d'exécution de simulations sur des composants

- Les HIOA sont composables par partage de variables continues et d'événements discrets or les modèles DEVS, par essence discrets, se composent uniquement par échange d'événements.
 - En DEVS, un phénomène continu doit être discrétisé à grain suffisamment fin pour assurer des calculs précis (ex.: intégration numérique par pas discrets).
 - De petits pas impliquent des échanges d'événements échantillonnant à haute fréquence les valeurs de variables partagées.
- Même à faible fréquence, les délais de communication entre composants répartis seraient très (trop) longs par rapport aux calculs effectués localement.
- D'où deux règles directrices :
 - 1 Que l'exécution de compositions de modèles DEVS issus d'HIOA partageant des variables continues soit *confinée à l'intérieur des composants*.
 - 2 Conséquemment, seule l'exécution de *compositions entre modèles issus de TIOA* puissent se faire *entre différents composants*.

Architectures DEVS sous BCM4Java

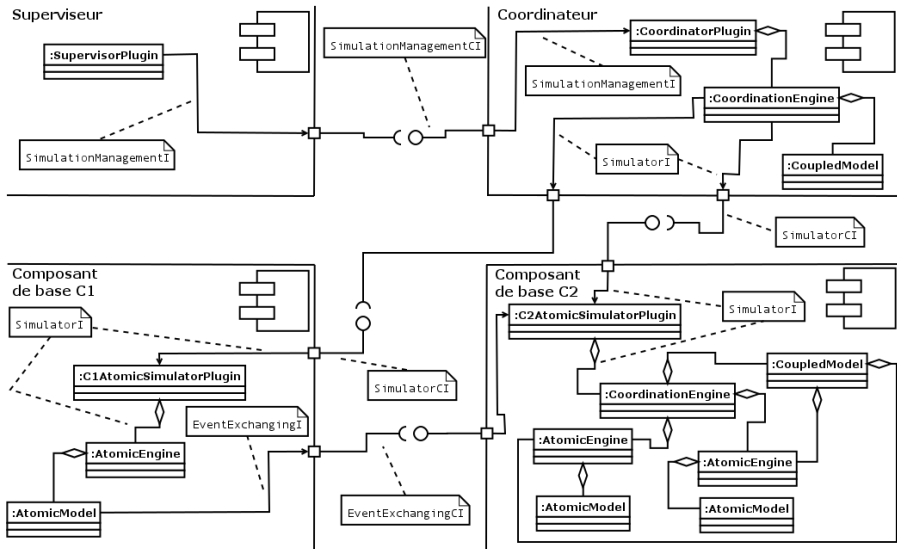
- Architecture en deux *niveaux principaux* :
 - une partie « basse » *décrites dans chaque composant* de base $\Rightarrow$ architectures locales ;
 - une partie « haute » formée de modèles couplés composant les précédentes $\Rightarrow$ architecture globale ;
 - Parties haute et basses sont décrites *séparément*, la partie haute voyant chaque partie basse composite comme un *modèle atomique*.
- Pour respecter l'abstraction du composant, tous les modèles de simulation d'un composant sont déployés sur ce composant.
 - Mais dans certains cas, on peut déroger pour exécuter certains modèles DEVS/HIOA sur des composants tiers *quand leur composition exige l'échange de variables continues*.
- Parce qu'il peut devenir utile d'utiliser différents simulateurs pour différents objectifs, il doit être possible de gérer plusieurs architectures globales et locales.



Exécution de simulations DEVS sous BCM4Java

- Comment définir et exécuter une architecture DEVS dans BCM4Java ?
 - ➊ Attacher des modèles DEVS aux composants, ce qui se fait par l'intermédiaire de greffons (*plug-ins*) réutilisables.
 - ➋ Interconnecter des modèles en passant par des connexions inter-composants *i.e.*, ports et connecteurs BCM4Java.
 - ➌ Gérer la création de simulateurs globaux puis l'exécution et la gestion de campagnes de simulations par des greffons de supervision qui vont abstraire l'utilisateur de ces considérations.
- On peut distinguer trois rôles à jouer :
 - ➊ Simulateur « unitaire » : associé à chaque composant de base.
 - ➋ Coordonnateur : exécutant un modèle couplé de l'architecture globale (partie haute).
 - ➌ Superviseur : assure
 - la création et l'interconnexion de l'architecture globale et
 - la gestion des campagnes de simulation (lancement, récupération puis analyse des résultats).

Schéma d'inter-connexion (partiel) DEVS via BCM4Java



Les greffons d'intégration DEVS/BCM I

- Outre la gestion des interfaces, ports et des connecteurs, les greffons vont recevoir les appels de méthodes entre moteurs et modèles situés dans différents composants.
 - Ils doivent donc implanter (au sens Java) les interfaces appropriées du *framework* DEVS pour jouer un rôle de *proxy* en relayant ces appels aux moteurs et modèles qu'ils contiennent.
Ex.: les greffons jouant le rôle de simulateurs (atomiques ou de coordination) doivent implanter l'interface `SimulatorI`.
- Ils doivent aussi créer l'architecture locale et interagir pour créer la partie de l'architecture globale.
 - Le greffon de simulation de modèles atomiques permet de créer des architectures de simulation locales (sous-arbres) puis utilise la fermeture de la composition pour présenter au reste de l'architecture ce sous-arbre local comme un modèle atomique.

Les greffons d'intégration DEVS/BCM II

- Le greffon de supervision connaît l'architecture globale (arborescence depuis la racine jusqu'aux racines des sous-arbres locaux vues comme des modèles atomiques) et l'instantiation de cette architecture crée les modèles couplés, leurs moteurs de coordination et leurs greffons sur les composants désignés.
- Enfin, outre les liens des modèles couplés vers leurs sous-modèles directs, le *framework* DEVS utilise certaines références Java dans les architectures de simulation pour interagir :
 - directement entre modèles atomiques par l'interface `EventsExchangingI`,
 - des sous-modèles vers leur modèle couplé parent par l'interface `ParentNotificationI`.

Ces connexions vont dans certains cas devoir se faire via des connexions entre composants (`EventsExchangingCI` et `ParentNotificationCI`), ce qui est aussi géré par les greffons.

Point de vue utilisateur : architectures locales

- ❶ Programmer les modèles de simulation MIL, SIL (voire HIL) locaux, avec la bibliothèque DEVS en Java.
- ❷ Créer une architecture locale, l'attacher au composant par un greffon de type simulation de modèle atomique, pour le présenter à l'architecture globale comme un modèle atomique.
 - Ceci produit une forme d'abstraction en boîte noire qui convient bien à la programmation par composants.
 - On masque l'architecture de simulation locale au sein du composant en créant les modèles/moteurs lors de la création du composant *i.e.*, lors de la création et de l'installation de son greffon.

Ex.: pour le composant WiFi de Molène, on aurait

```
Architecture localArchitecture =  
    new Architecture(WiFiModel.URI, atomicModelDescriptors,  
                    coupledModelDescriptors, TimeUnit.SECONDS) ;  
AtomicSimulatorPlugin asp = new AtomicSimulatorPlugin() ;  
asp.setPluginURI(WiFiModel.URI) ;  
asp.setSimulationArchitecture(localArchitecture) ;  
this.installPlugin(asp) ;
```

Architectures de simulation combinées DEVS/BCM4Java

- Pour décrire un modèle dans l'architecture globale, on ajoute à son descripteur l'URI du port de reflexion entrant du composant qui le possède et où on va devoir le créer.
 - Les composants qui possèdent une architecture locale sont décrits dans l'architecture globale comme des modèles atomiques, masquant ainsi les détails de cette architecture locale.
 - Les composants coordinateurs, qui vont détenir des modèles couplés, n'ont pas à créer explicitement de greffon lors de leur création ; ils vont être créés lors du processus d'instantiation de l'architecture globale.
- Du point de vue de l'utilisateur, peu de changements par rapport au *framework* DEVS en Java ; ce sont les descripteurs spécifiques à l'intégration et les greffons qui implantent l'algorithme d'instantiation croisée DEVS/BCM4Java.
- L'ensemble des ajouts aux composants, des greffons, des ports et connecteurs spécifiques à cette intégration forment une extension de BCM4Java appelée BCM4Java-CyPhy.

Exemple WiFi I

- Dans l'exemple Molène, le modèle est détenu par un composant WiFi et son descripteur vu comme un modèle atomique dans l'architecture globale est créé par :

```
ComponentAtomicModelDescriptor.create(  
    WiFiModel.URI,          // URI du modèle  
    null,                   // pas d'événements importés  
    (Class<? extends EventI>[])  
        new Class<?>[]{WiFiBandwidthReading.class, // événements exportés  
                        InterruptionEvent.class,  
                        ResumptionEvent.class},  
    TimeUnit.SECONDS,       // unité de temps  
    // URI du port de réflexion entrant du composant hôte  
    // ici, on les a préalablement mises dans une "map"  
    modelURIs2componentURIs.get(WiFiModel.URI))
```

- Les autres modèles « atomiques » décrits, le modèle couplé global appelé MoleneModel est décrit par :

Exemple WiFi II

```
ComponentCoupledModelDescriptor.create(  
    MoleneModel.class,      // classe à instancier pour créer le modèle  
    MoleneModel.URI,       // URI du modèle à créer  
    submodels,              // ensemble des URIs de ses sous-modèles  
    null,                   // pas d'événements importés  
    null,                   // pas d'événements exportés  
    connections,            // connexions exportation/importation  
    null,                   // pas de fabrique, fabrique standard  
    SimulationEngineCreationMode.COORDINATION_ENGINE, // moteur  
    // URI du port de réflexion entrant du composant hôte  
    modelURIs2componentURIs.get(MoleneModel.URI))
```

- Et enfin, l'architecture et le greffon superviseur sont créés par :

```
ComponentModelArchitecture architecture =  
    new ComponentModelArchitecture(  
        MoleneModel.URI,          // URI du modèle couplé racine  
        atomicModelDescriptors,    // descripteurs des modèles «~atomiques~»  
        coupledModelDescriptors,  // descripteurs des modèles couplés  
        TimeUnit.SECONDS) ;       // unité de temps de simulation  
this.sp = new SupervisorPlugin(architecture) ;  
sp.setPluginURI("supervisor") ;  
this.installPlugin(this.sp) ;    // dans le composant superviseur hôte
```

Création et exécution

- À l'exécution, le composant superviseur crée l'architecture de simulation globale, instancie le greffon superviseur en lui passant cette architecture en paramètre, ce dernier crée le simulateur global en appelant la méthode `constructSimulator` puis lance la simulation en appelant ici la méthode `startRTSimulation`. Exemple : en supposant que la variable `sp` contient la référence au greffon de supervision

```
sp.createSimulator();  
this.sp.startRTSimulation(System.currentTimeMillis() + 1000, 0.0, 5000.0);
```

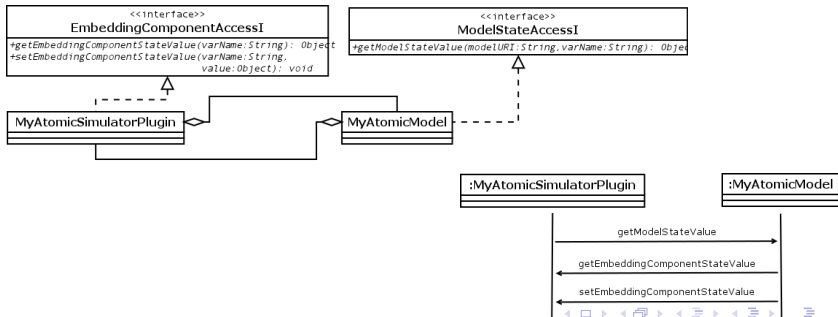
- Avec la possibilité d'utiliser comme dans le *framework* DEVS le protocole de passage de paramètres d'exécution par la méthode `setSimulationRunParameters`.
- *Nota* : la méthode `startRTSimulation` réalise une simulation *en temps réel* permettant des échanges avec le logiciel selon une même référence temporelle. Nous y revenons...

Embryon de composant cyber-physique : BCM4Java-CyPhy

- BCM4Java-CyPhy offre une classe abstraite `AbstractCyPhyComponent` dont doivent hériter les classes définissant les composants cyber-physiques qui définit :
 - une table de hachage liant l'URI du modèle racine de chaque architecture de simulation locale définie par le composant ;
 - deux tables de hachage liant les URIs des architectures globales de simulation aux URIs des modèles racines des architectures locales et l'inverse ;
 - une extension aux interfaces de réflexion pour les composants cyber-physiques qui offre des méthodes permettant de récupérer les URIs de certains ports spécifiques à la simulation DEVS pour faciliter la composition des modèles de simulation entre compsoants.
- D'autres éléments devront être ajoutées pour améliorer l'intégration BCM4Java/DEVS, définir les activités de déploiement (voir ci-après), utiliser des interfaces riches, spécifier les propriétés d'adaptation, etc.

Échanges de données composants/modèles

- Dans les simulations SIL, il est généralement nécessaire d'être capable de communiquer entre composants hôtes et modèles :
 - le composant peut avoir besoin d'accéder une valeur de variable d'un modèle qu'il contient ;
 - le modèle peut vouloir accéder à une donnée du composant.
- Protocole des interfaces `ModelStateAccessI` et `EmbeddingComponentStateAccessI` :



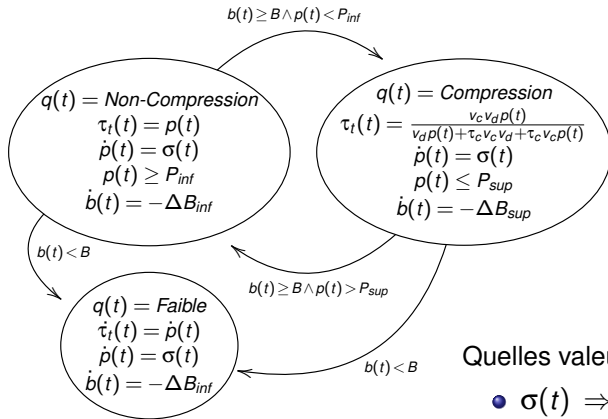
Déclenchement d'exécution de code composants/modèles

- Modèle vers composant :
 - Un modèle peut récupérer la référence sur son composant hôte grâce au protocole `setSimulationRunParameters` puis appeler ses méthodes ; le greffon de simulation atomique ajoute cette référence automatiquement aux paramètres.
 - Attention à respecter les exclusions mutuelles du composant hôte car les activités de simulation sont ordonnancées sur des *threads* distincts de ceux utilisés par ce dernier.
- Composant vers modèle :
 - Cohérent seulement si simulation temps réel.
 - Le greffon `RTAtomicSimulatorPlugin` servant à gérer les simulations temps réel offre la méthode :
`triggerExternalEvent(String destinationModelURI, EventFactoryFI ef)`
 - `destinationModelURI` : URI d'un modèle receveur
 - `ef` : λ prenant en paramètre le temps simulé courant pour créer un événement devant être reçu et exécuté à cet instant.

Plan

- 1 Opérations d'adaptation
- 2 Les actionneurs et leurs interfaces
- 3 Test et validation des entités adaptables
- 4 Simulateurs DEVS et composants BCM4Java
- 5 Aide au déploiement

Problématique — retour sur notre exemple Molène



Quelles valeurs pour :

- $\sigma(t) \Rightarrow$ mesures de terrain
- $P_{sup}, P_{inf}, B \Rightarrow$ choix
- $v_c, v_d, \tau_c, \Delta B_{inf}, \Delta B_{sup} ???$
- durée des adaptations ???

Contrôle et identification du système

- Dans les CPS, la plupart des paramètres définissant les modèles de comportement dépendent du contexte de déploiement : performance du matériel, outils logiciels (ex.: quels outils de compression ?), etc.
- Pour les estimer de manière suffisamment précise pour établir un bon modèle de contrôle, il faut connaître ce(s) contexte(s). Deux options sont alors possibles :
 - 1 identifier au développement les contextes possibles et estimer ces paramètres à l'avance sur ces contextes pour les spécifier comme paramètres de configuration au préalable, ou
 - 2 attendre au déploiement et les mesurer *in situ* avant de commencer l'exécution ou pendant l'exécution pour (re)configurer au déploiement ou dynamiquement.
- Dans tous les cas, pour estimer ces paramètres, il faudra instrumenter l'entité adaptable pour ce faire.

Limites de la modélisation *amont* pour l'adaptabilité

- Le génie logiciel classique prévoit de modéliser en amont du déploiement pour des développements produisant du logiciel déployable sur des plates-formes très variées.
- Mais à cette étape, on ne peut établir qu'un modèle comportemental générique, puisque ses paramètres précis dépendent du contexte de déploiement logiciel et matériel (p.e., performance).
Ex.: $p(k+1) = ap(k) + bu(k)$, Cours 6.
- Ce modèle générique doit ensuite être identifié pour donner un modèle *concret*, c'est-à-dire que des valeurs précises pour tous les paramètres, coefficients, etc. sont déterminées à partir de mesures faites *dans le contexte de déploiement*.
Ex.: $p(k+1) = 0,43p(k) + 0,47u(k)$, Cours 6.
- Enfin, pour procéder aux tests unitaires et d'intégration, puis valider le système, il faudra *simuler* tout ou partie des modèles fonctionnels et comportementaux.

Auto-caractérisation et auto-configuration

- Automatiser ces fonctionnalités :
 - Proposer des procédures prédéfinies permettant d'estimer les caractéristiques de l'entité, de ses mesures, de ses opérations d'adaptation et de l'impact des opérations d'adaptation sur les propriétés adaptables.
 - Proposer une interface pour récupérer ces données, qui pourra être utilisées par l'entité de contrôle de manière à configurer son propre modèle et sa loi de commande.
- En réalité, cette idée poursuit le même but que certaines approches de contrôle adaptatif ou d'apprentissage automatique : estimer les paramètres du modèle de comportement de l'entité adaptable pour obtenir un modèle de contrôle correct et efficace.
- Un lien intéressant peut aussi être fait avec l'approche *BITE* pour *Built-In Test Equipment* de plus en plus utilisée dans l'industrie pour superviser les appareils et détecter les anomalies en cours de fonctionnement.

Récapitulons...

- 1 La capacité d'adaptation exigée dans les application réelles dépasse la simple juxtaposition d'opérations d'adaptation : il faut en **superviser l'exécution**, les organiser comme **transactions**, et les **coordonner** avec d'autres adaptations.
- 2 Pour être réellement contrôlable, une entité adaptable doit savoir **caractériser de la manière la plus précise possible les effets** des adaptations sur les mesures des capteurs pour appliquer la bonne loi de contrôle et les exprimer comme méta-propriétés.
- 3 Concevoir une entité adaptable dans un contexte **ouvert** est **difficile**, et ne peut être réalisé sans une **explicitation** complète de ses propriétés.
- 4 La validation par le test pouvant rarement se faire systématiquement dans tous les contextes de déploiement, la **simulation** est un outil essentiel à la mise au point des entités adaptables ce qui mène au concept de **composants simulables** comme pierre angulaire des systèmes cyber-physiques et autonomiques et de leur méthodologie de développement.
- 5 La plupart des propriétés non-fonctionnelles des entités adaptables ne peuvent être **quantifiées** que dans le contexte de déploiement ; il est donc très utile de disposer de moyens **automatiques** pour **identifier** le modèle de comportement de l'entité au moment de son **déploiement**.

Pour aller plus loin : sélection de lectures recommandées

- *Autonomic Computing*, P. Lalande, J. McCann et A. Diaconescu, Springer-Verlag, 2013, chapitres 5 et 6.
- X. Dutreilh *et al.*, *From Data Center Resource Allocation to Control Theory and Back*, CLOUD 2010, juillet 2010.
- *Theory of Modeling and Simulation*, B.P. Zeigler et al., 2^e édition, Academic Press, 2000.